

CS & IT ENGINEERING

DATA STRUCTURE

Binary Tree (Part-III)



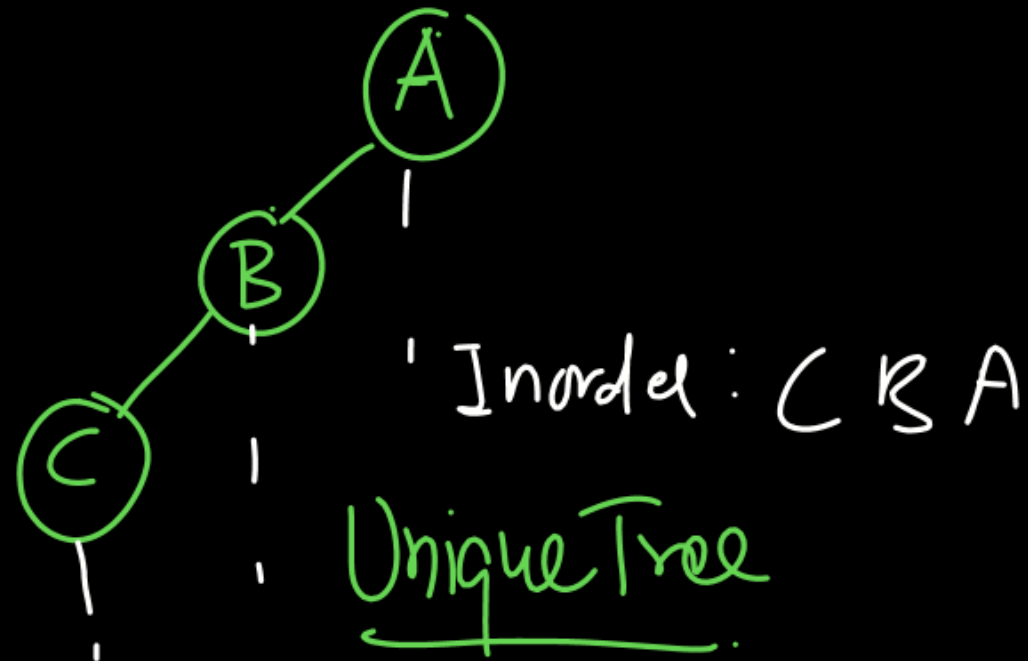
JOIN TELEGRAM TO ENROLL

SHAIENDRA SIR

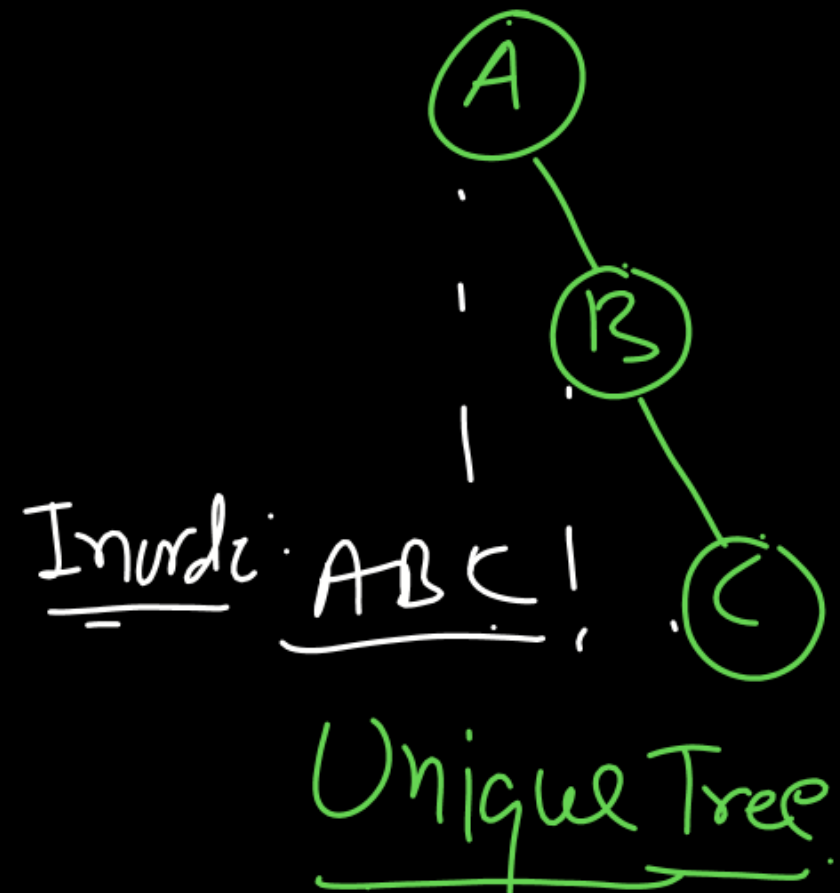
Agenda for Today

- Binary Tree Traversal
- Binary Tree Construction
- Pseudo code Questions

Case 4:- $\xrightarrow{\text{Root Left Right}}$ Preorder: ABC
 Inorder: CBA (or)
 $\xrightarrow{\text{Left Root Right}}$



Case 5: $\xrightarrow{\text{Left Right Root}}$ Postorder: CBA
 Inorder: ABC



NOTE:- if (Preorder) or (Postorder) given Then No. of Binary Tree = 1
 i.e. Unique Binary Tree

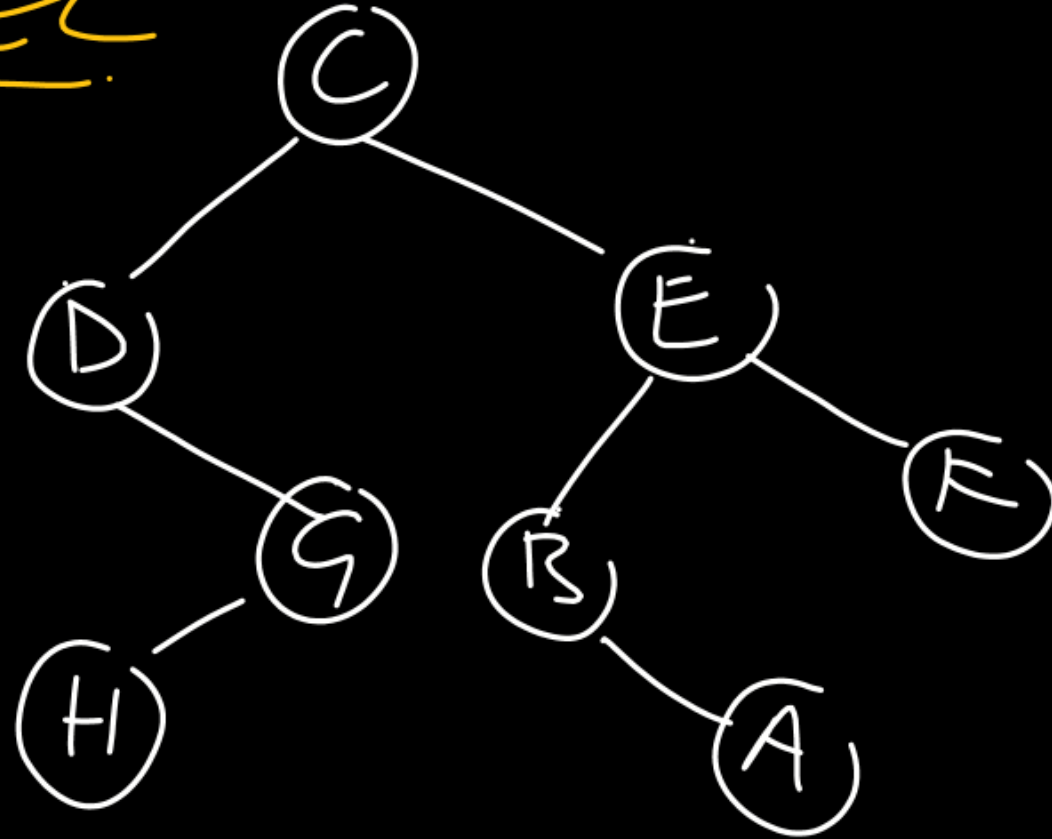
Question:- Pre: C D G H E B A F

IN: D H G C B A E F

Postorder: ? H G D A B F E C

leaf nodes = ?

H, A, F



Question:-

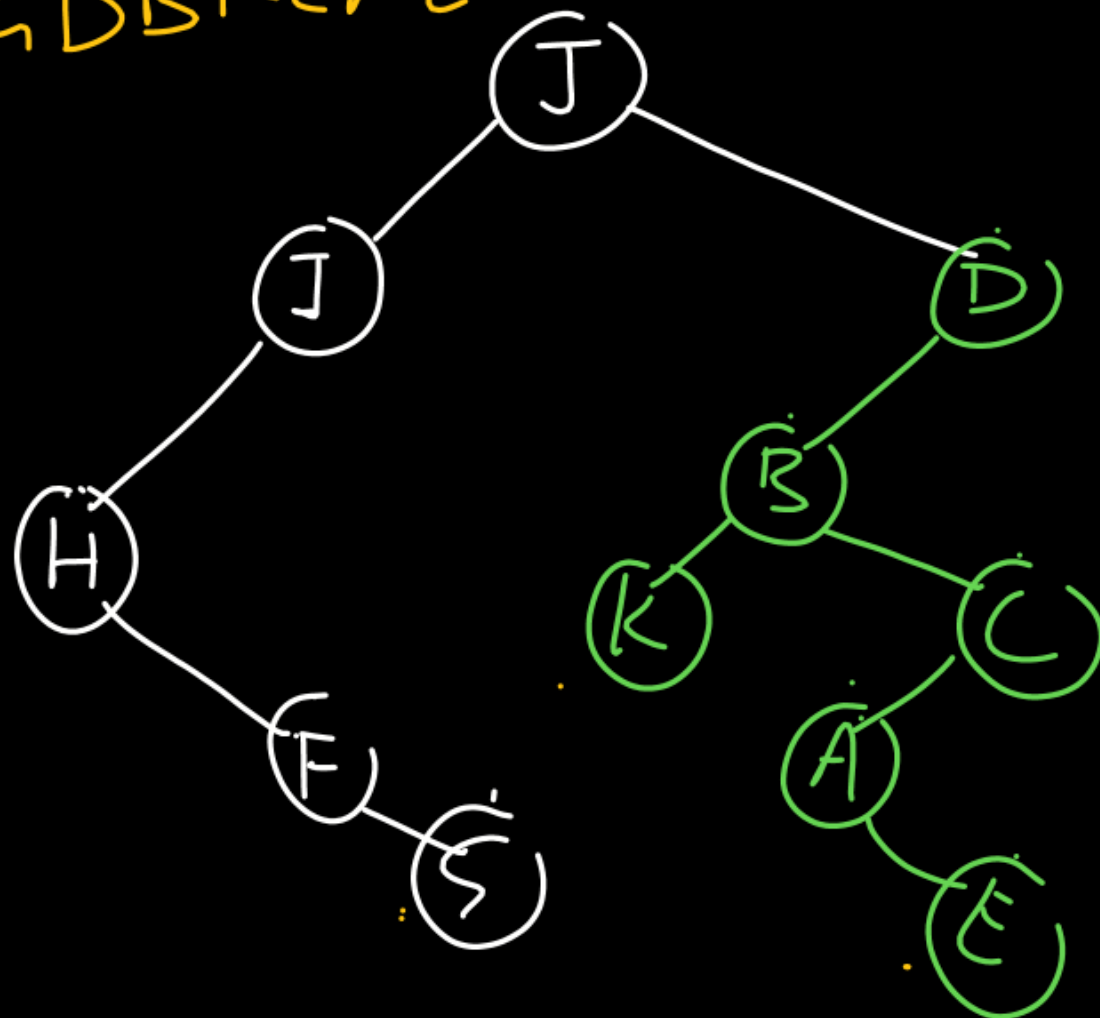
Post: G F H I K E A C B D J $\Rightarrow n$

IN: H F G I J K B A E C D $\Rightarrow n$

Preorder? J I H F G D B K C A E

Worst case TC:-?

leaf nodes? G K E



Time Complexity :-

Pre: $(A) (B) C D E \Rightarrow n$

IN: E D C B A $\Rightarrow n$

for A	→	n
" B	→	n-1
C	→	n-2
⋮	⋮	⋮
⋮	⋮	2
⋮	⋮	1

$$\Rightarrow 1 + 2 + 3 + \dots + n$$

$$\Rightarrow \frac{\eta(n+1)}{2} \Rightarrow \underline{O(n^2)}$$

NOTE:- Preorder and Inorder (or) Postorder and Inorder given then Unique Binary Tree Construction Possible with $O(n^2)$ time, if Inorder sorted (BST) Then $O(n \log_2 n)$

Binary Tree Construction:-

Construct Binary Tree of

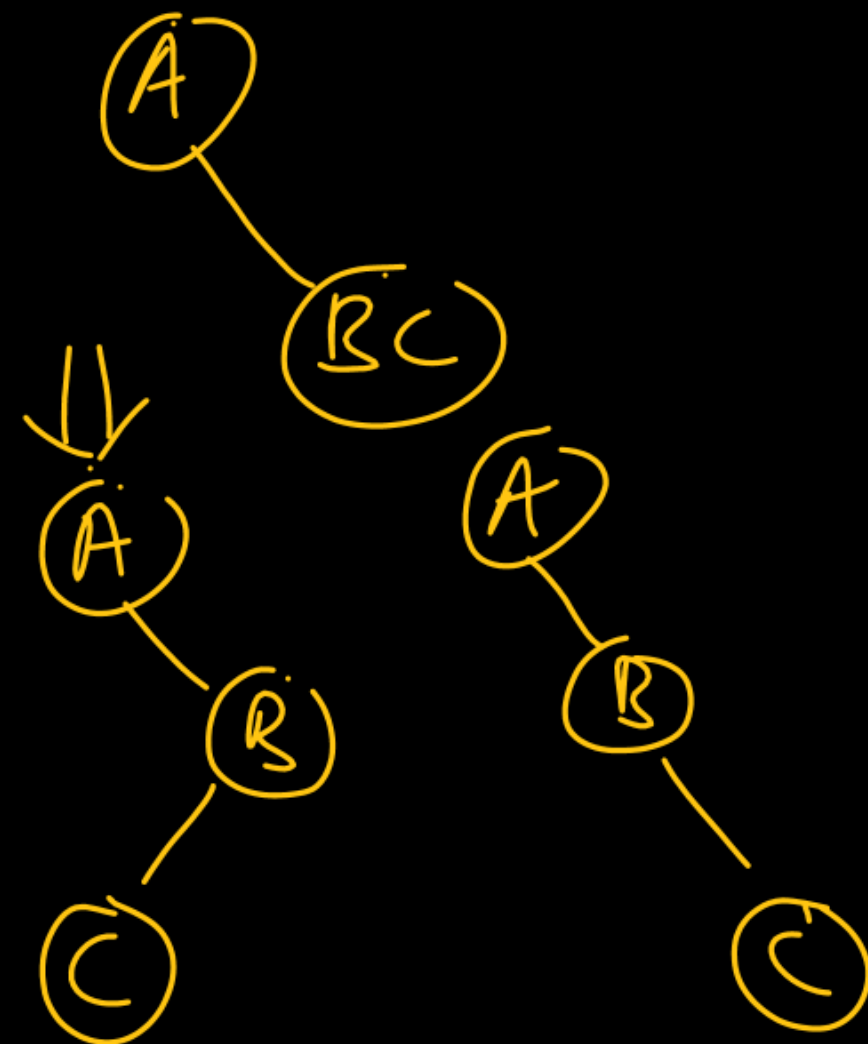
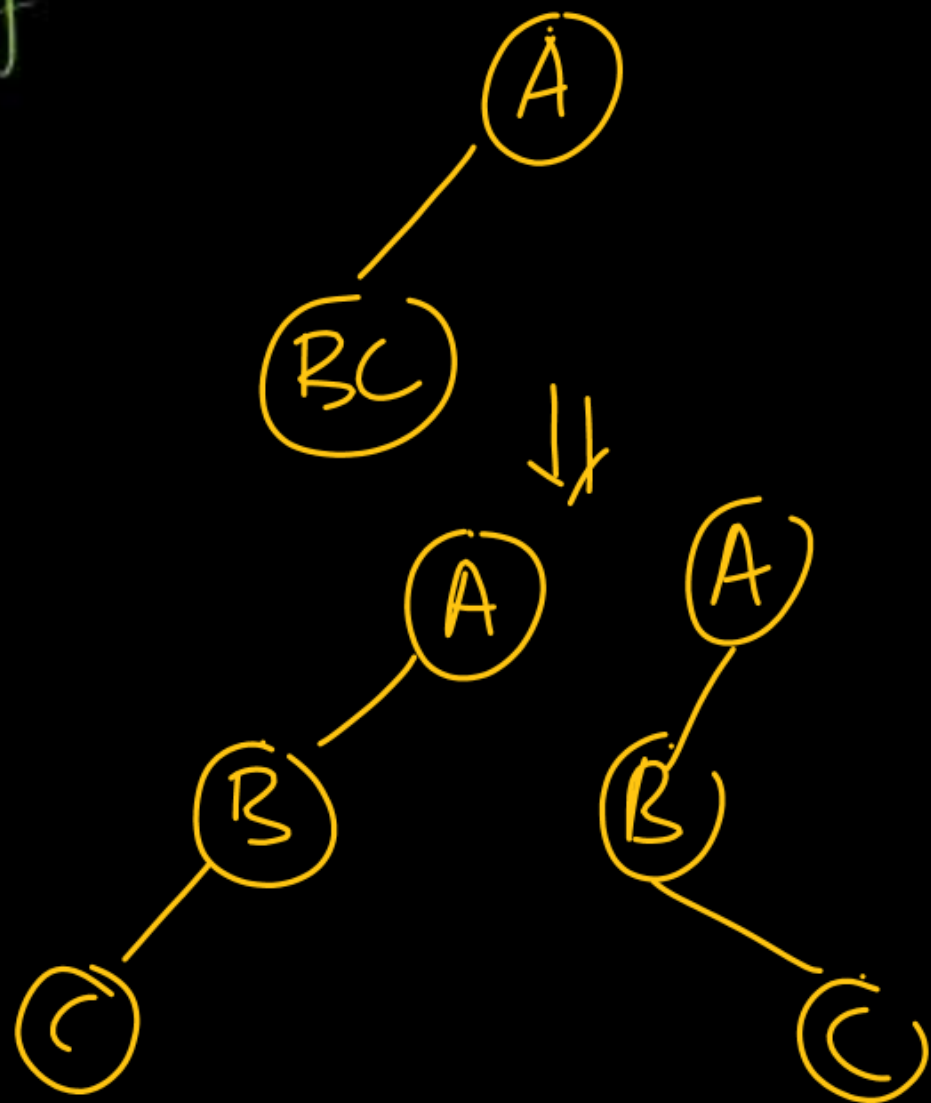
pre-order: A B C

Post-order: C B A

Root left Right

Pre: A B C

: A BC



Binary Tree Construction:-

Construct Binary Tree of

Pre-order: A B C

Post-order: C B A

In: C B A

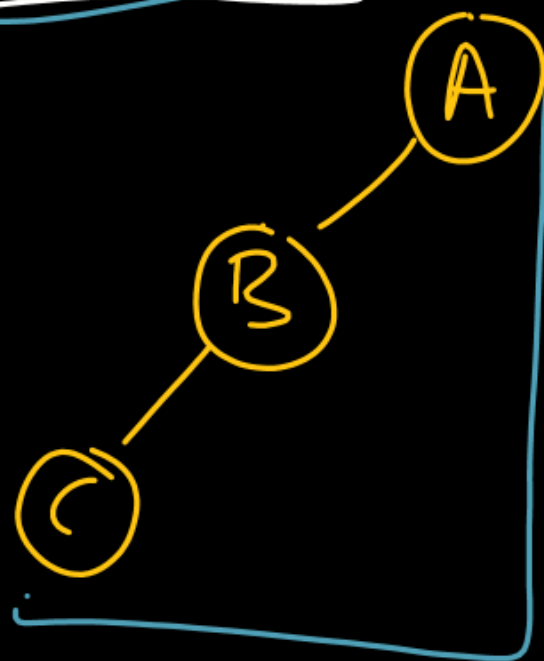
Pre: A B C

Post: C B A

Inorder = ?

$$2^{n-1}$$

$$T.C = O(2^n)$$



In: BCA

Pre: ABC

Post: CBA



In: ACB

Pre: ABC

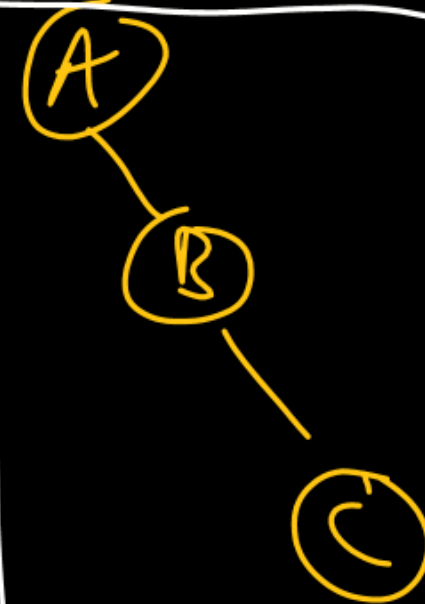
Post: CBA



In: ABC

Pre: ABC

Post: CBA

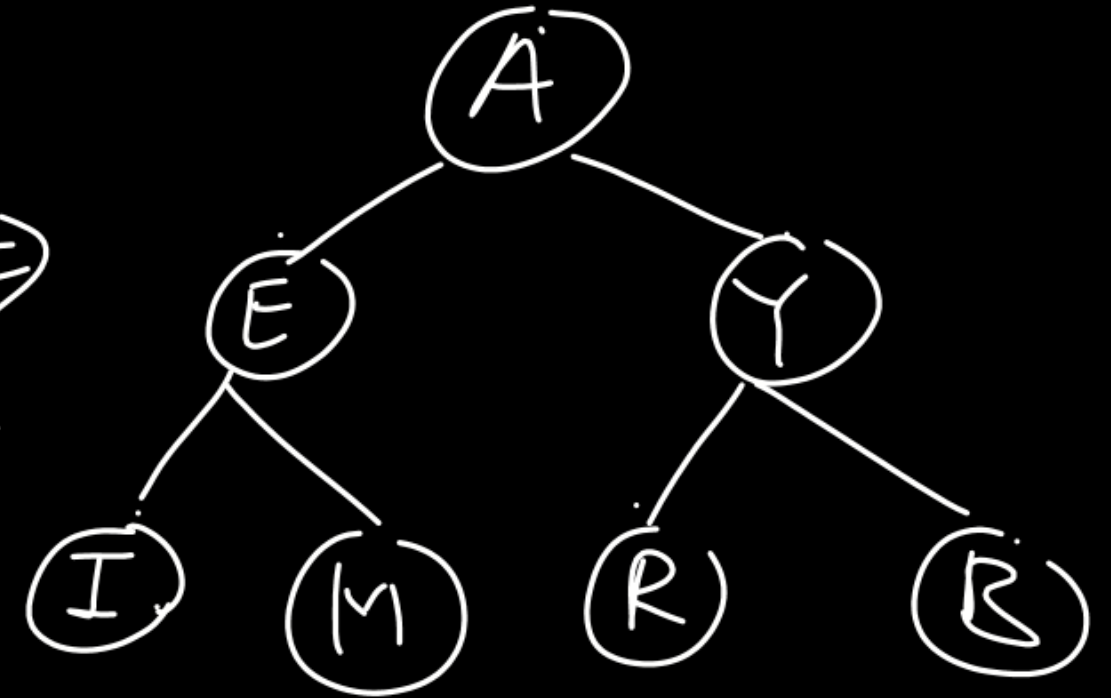
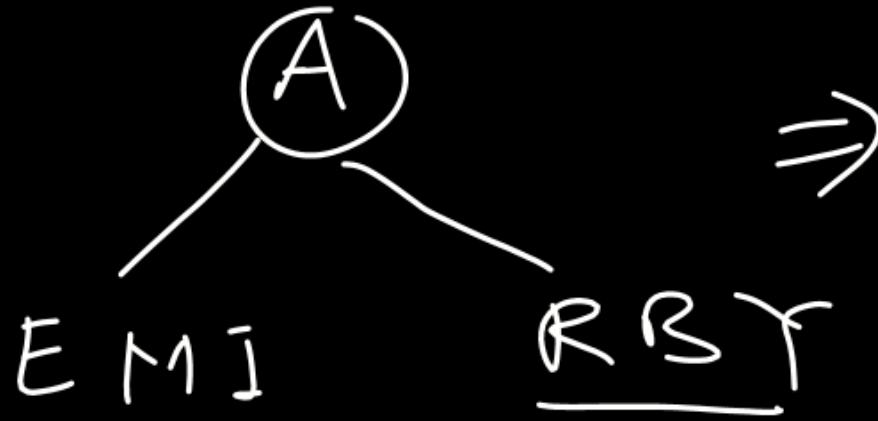


Question:- Construct Full Binary Tree :- $\rightarrow$ 0 or 2 child

Pre-order: A E I M Y R B
Post-order: I M E R B Y A

Inorder: I E M A R Y B

leaf nodes: I M R B



NOTE: If preorder and postorder traversals are given, you can construct the unique full binary Tree

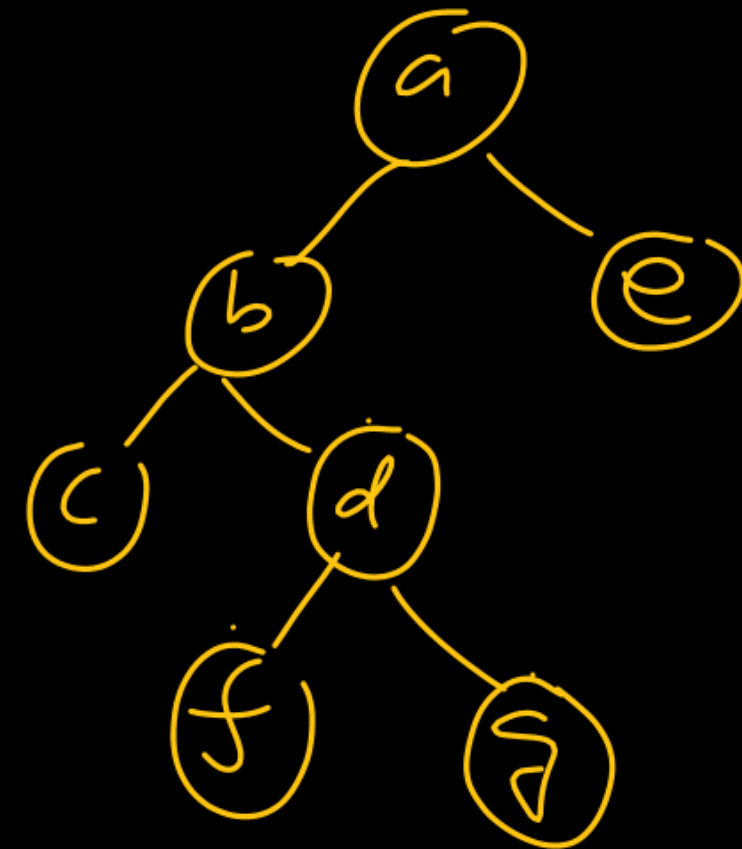
Question:- Construct full Binary Tree

Pre-order: (a) (b) c (d f g) e

Postorder: c (f g d) b e (a)

Inorder: ?

Leaf nodes: ?

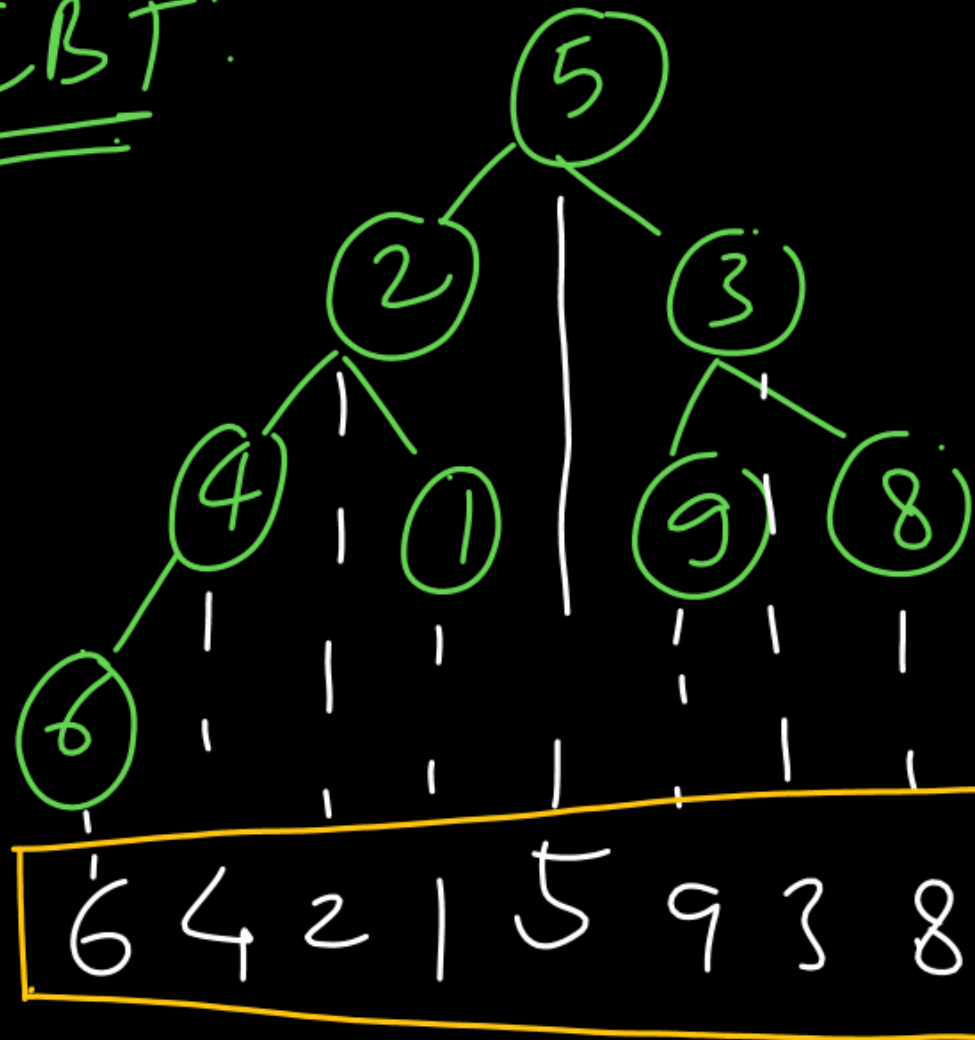


Question:- Construct Complete Binary Tree using

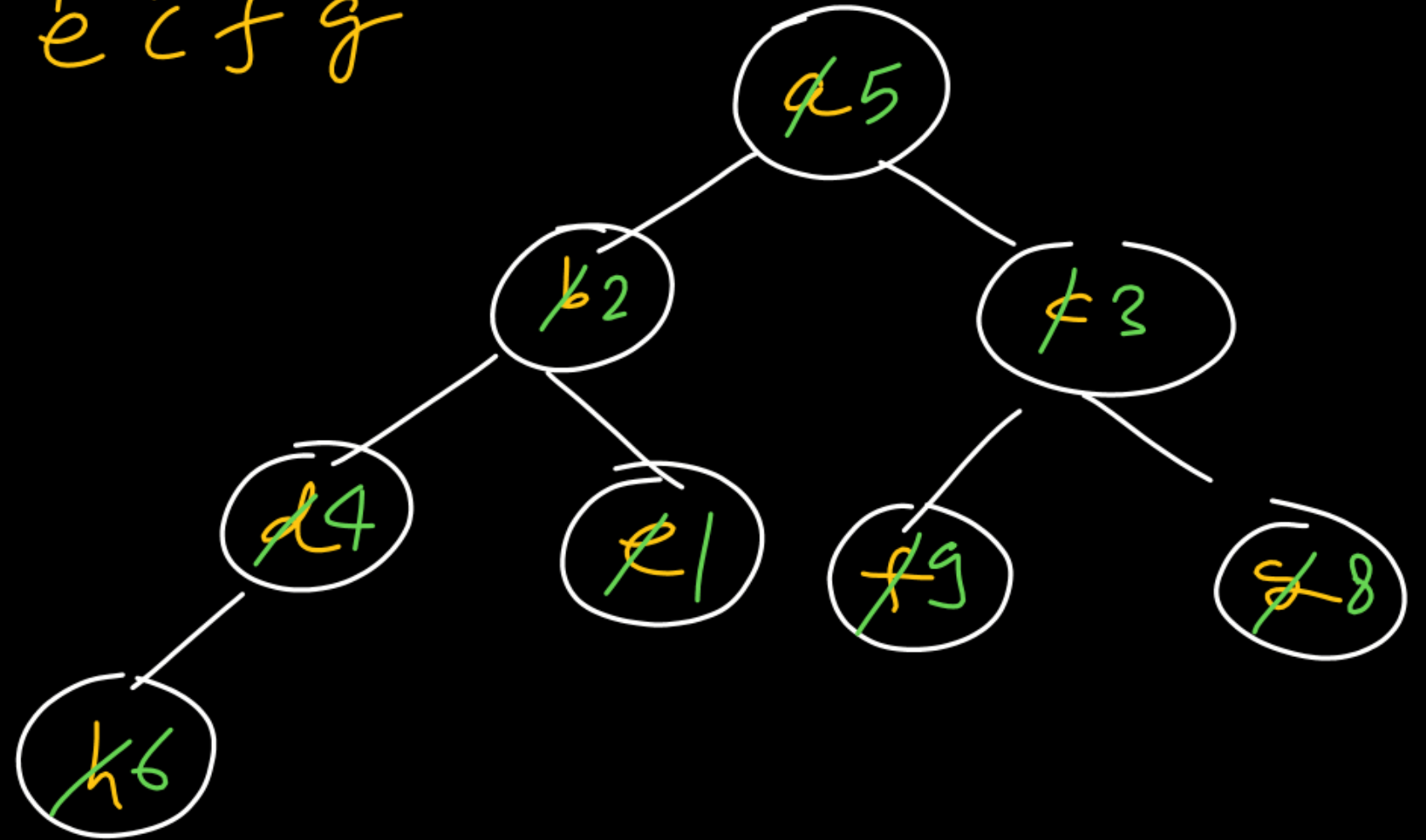
Pre-order: 5 2 4 6 1 3 9 8

preorder: a b d h e c f g

CBT:



Inorder



NOTE:-

1) General Binary Tree: Inorder must

for Unique BT $\Rightarrow \begin{cases} \text{Inorder} + \text{Pre-order} \\ \text{(or)} \\ \text{Inorder} + \text{Postorder} \end{cases}$

2) Full Binary Tree:- any two are enough for Unique Full Binary Tree.

for Unique Full BT $\Rightarrow \begin{cases} \text{Inorder} + \text{Pre-order} \\ \text{(or)} \\ \text{Inorder} + \text{Postorder} \\ \text{(or)} \\ \text{Preorder} + \text{Postorder} \end{cases}$

3) Complete Binary Tree:- one of the any order is enough for Uniqueness

for Unique Complete BT $\Rightarrow \text{Inorder (or) Postorder (or) Preorder}$

Height of Binary Tree:

```
int H(node *t)
```

```
{ if(!t)
```

```
    return 0;
```

```
if(!(t->left) && !(t->right))
```

```
    return 0;
```

```
int l, r;
```

```
l = H(t->left);
```

```
r = H(t->right);
```

```
return (1 + (l > r ? l : r));  
}
```

$$H(T) = \begin{cases} 0 & , \text{ if } T \text{ is Null} \\ 0 & , \text{ if } T \text{ is leaf} \\ 1 + \max(H(LST), H(RST)) & \text{ otherwise} \end{cases}$$

Count No of Nodes :-

```
int NN(node *t)
```

```
{ if (t)
```

```
{ int l, r;
```

```
l = NN(t->left);
```

```
r = NN(t->right);
```

```
return (1+l+r);
```

```
}
```

```
} return 0;
```

$$NN(T) = \begin{cases} 0, & \text{if } T \text{ is NULL} \\ 1 + NN(LST) + NN(RST), & \text{otherwise} \end{cases}$$

Count No of Leaf node

$$LN(T) = \begin{cases} 0 & , \quad \text{if } T \text{ is Null} \\ 1 & , \quad \text{if } T \text{ is Leaf} \\ LN(LST) + LN(RST), & \text{Otherwise} \end{cases}$$

Count Internal nodes (non leaf nodes):

$$NI = \begin{cases} 0 & , \quad \text{if } T \text{ is NULL.} \\ 0 & , \quad \text{if } T \text{ is Leaf} \\ 1 + NI(LST) + NI(RST), & \text{otherwise} \end{cases}$$

Count full nodes (2 child nodes) :-

```
int FN(node *t)
```

```
{ if (!t) return 0
```

```
  if (! (t->left) && ! (t->right))
```

```
    return 0
```

```
  return ( FN(t->left) + FN(t->right) + ((t->left) && (t->right)) ? 1 : 0 );
```

```
}
```

$$FN(T) = \begin{cases} 0 & , \text{ if } T \text{ is NULL.} \\ 0 & , \text{ if } T \text{ is Leaf.} \\ 1 + FN(LST) + FN(RST), & \text{ if } T \text{ is Full node.} \\ \underline{FN(LST) + FN(RST)}, & \text{ Otherwise} \end{cases}$$

Question: Analyze the given function.

```
int xyz(node *t)
```

```
Σ if (t == NULL) return 1;
```

```
if((t->left==NULL)&&(t->right==NULL))
    return 1;
```

if ((t->left != NULL) && (t->right != NULL))

return ((xyz(t → left) && (xyz(t → right)),

else

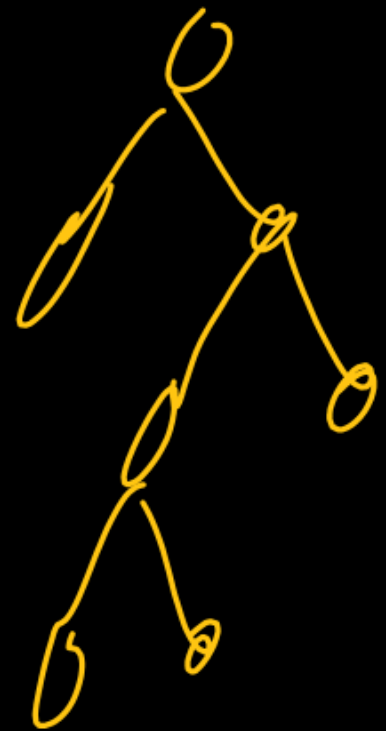
return 0;

3

Inters

$\frac{1}{\sqrt{2}}$

Definieren 0



SBT



FBT

Question: Analyze the given function.

```
int func(node *t1, node *t2)
{
    if (t1 == NULL && t2 == NULL)
        return 1;
```

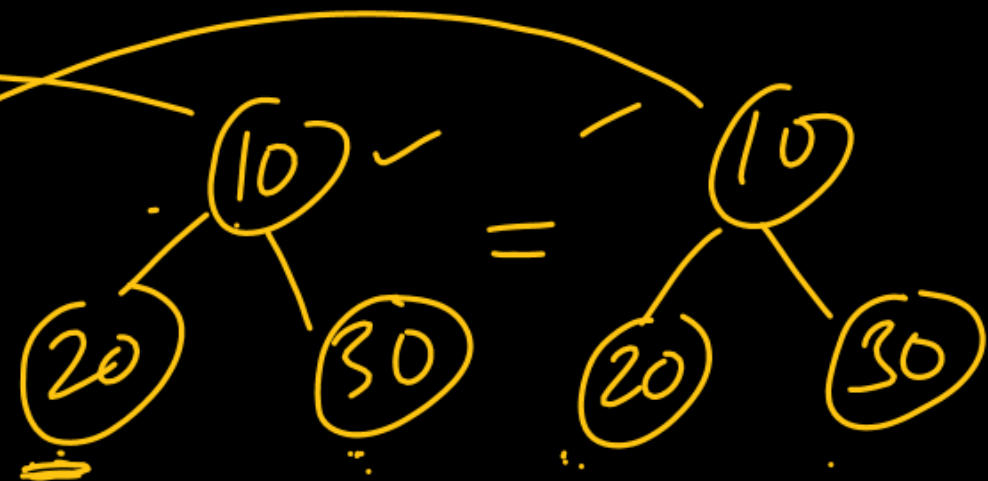
```
    if ((t1 == NULL && t2 != NULL) || (t1 != NULL && t2 == NULL))
        return 0;
```

```
    => if (t1->data == t2->data)
```

```
        return (func(t1->left, t2->left) && func(t1->right, t2->right))
```

```
    else
        return 0
```

```
}
```



Question: Analyze the given function.

```
void do(node *t)
```

```
{ if(t)
```

```
{ do(t->left);  
  do(t->right);
```

```
  node *temp;
```

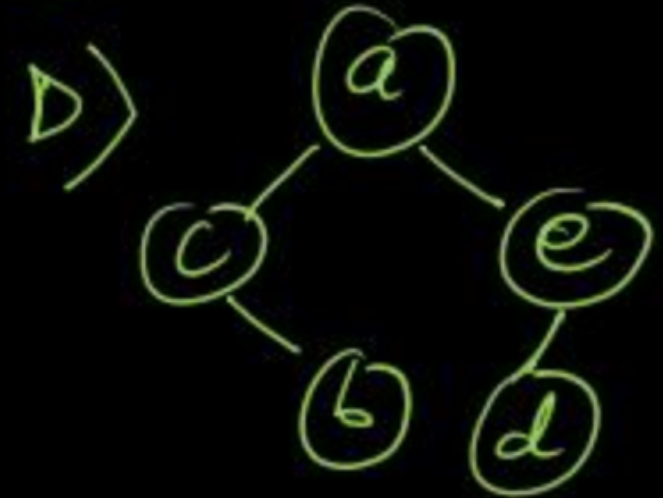
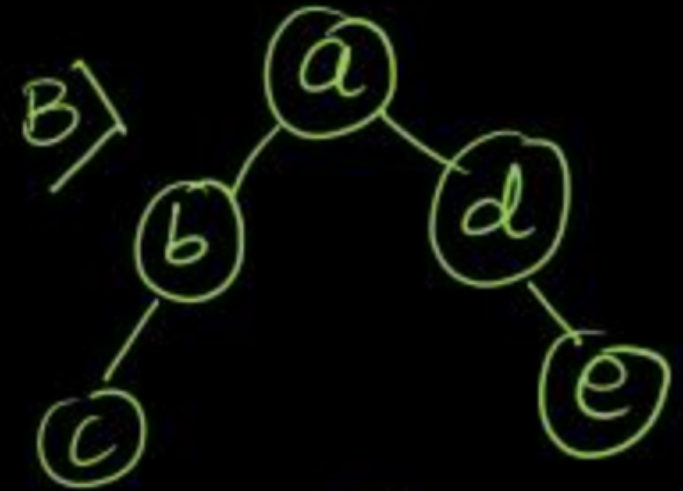
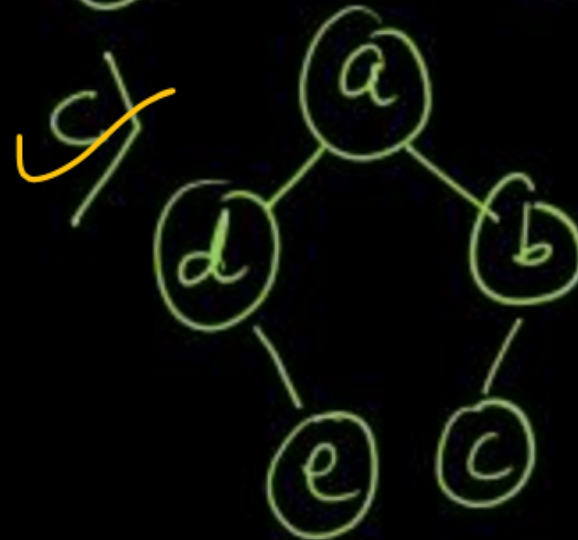
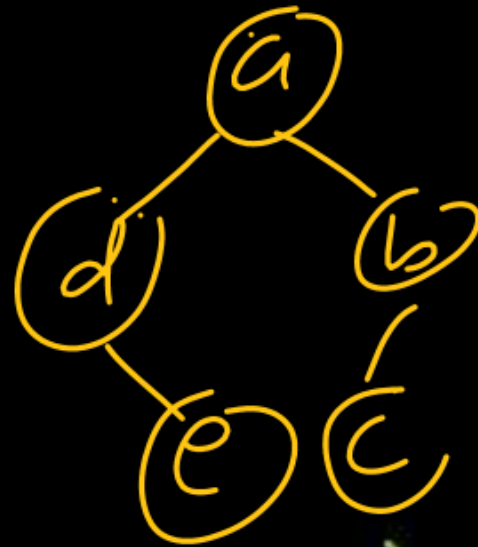
```
  temp = t->right;
```

```
  t->right = t->left;
```

```
  t->left = temp;
```

```
}
```

```
}
```



Question: Analyze the given function.

```
node *func (node *root)
```

```
{ node *temp;
```

```
if (!root) return NULL;
```

```
EnQueue (root);
```

```
While (!IsEmpty Queue())
```

```
{ temp = DeQueue();
```

```
if (temp->left)
```

```
EnQueue (temp->left);
```

```
if (temp->right)
```

```
EnQueue (temp->right);
```

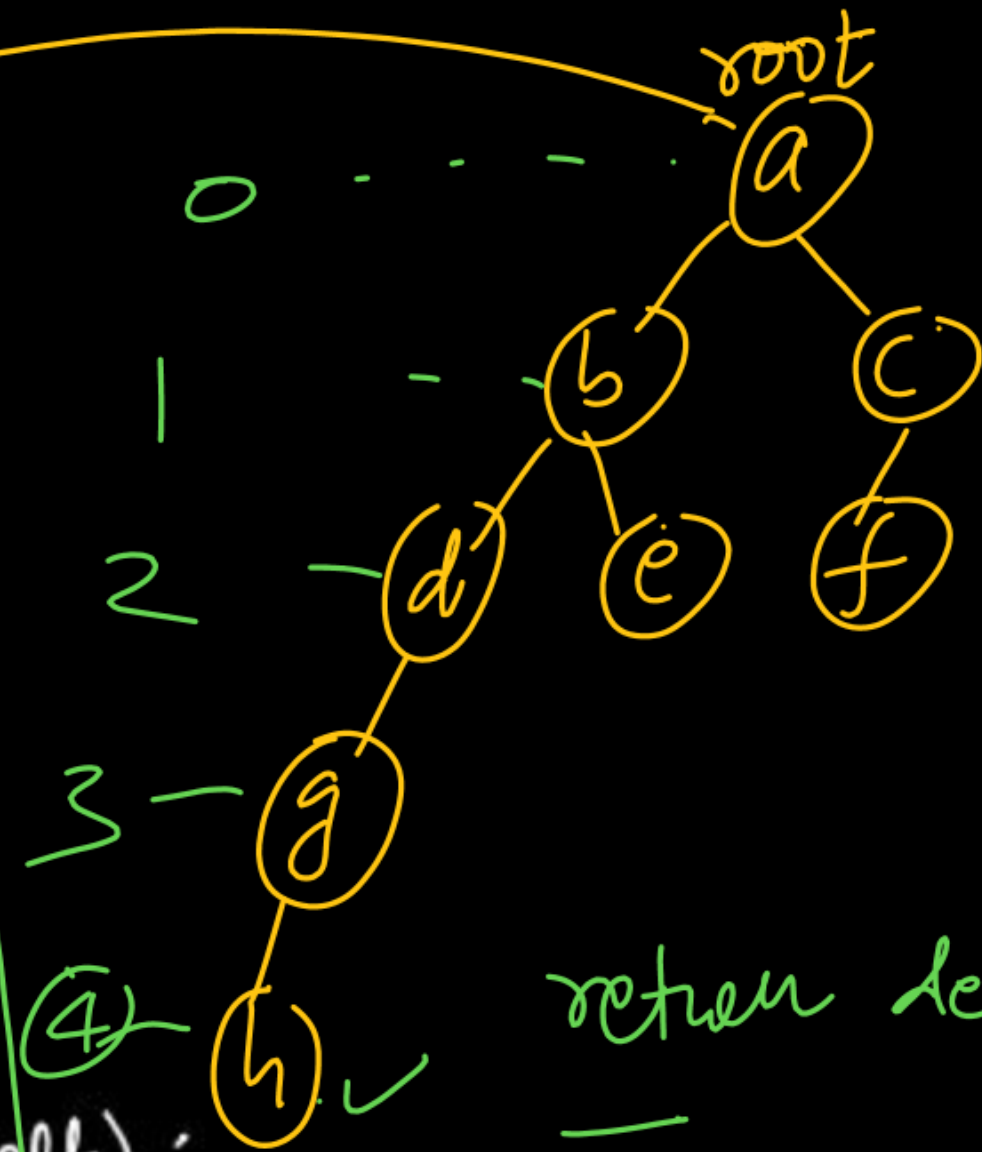
```
}  
return temp;
```

```
}
```

~~temp = root~~



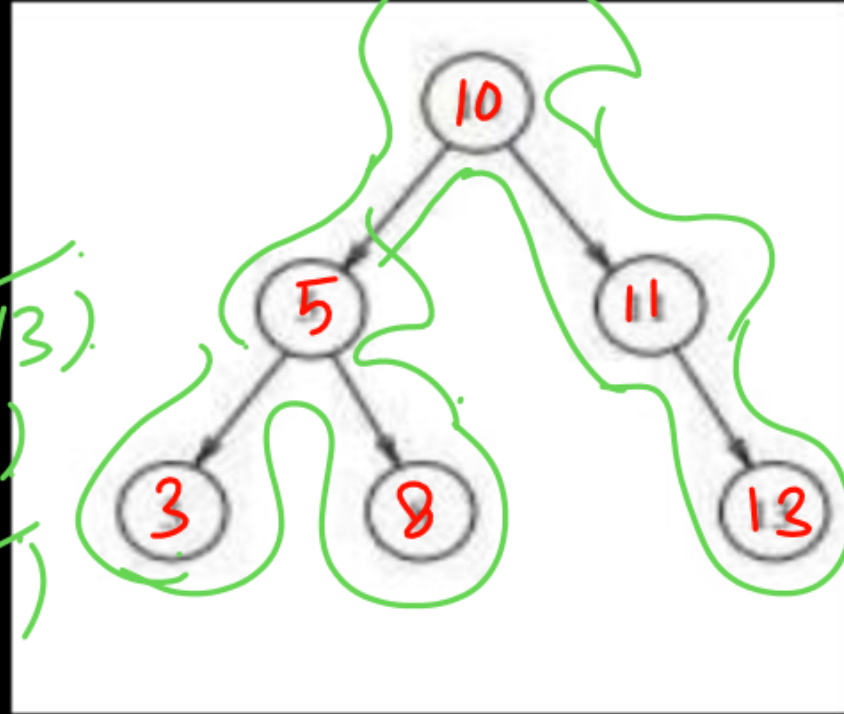
temp = (h)



Q) Consider the C function foo and the binary tree shown.

```
typedef struct node {
    int val;
    struct node *left, *right;
} node;
```

```
int foo(node *p) {
    int retval;
    if (p == NULL)
        return 0;
    else {
        retval = p->val + foo(p->left) + foo(p->right);
        printf("%d ", retval);
        return retval;
    }
}
```



$$\begin{aligned} \text{foo}(10) &= 10 + \text{foo}(5) + \text{foo}(11) \\ \text{retval} \quad & 10 + 16 + 24 = 50 \end{aligned}$$

$$\begin{aligned} \text{foo}(5) &= 5 + \text{foo}(3) + \text{foo}(8) \\ \text{retval} \quad & 3 + 8 \end{aligned}$$

$$\begin{aligned} \text{foo}(11) &= 11 + 0 + \text{foo}(13) \\ \text{retval} \quad & 11 + 13 = 24 \end{aligned}$$

$$\begin{aligned} \text{foo}(3) &= 3 + 0 + 0 \\ \text{retval} &= 3 \end{aligned}$$

$$\begin{aligned} \text{foo}(8) &= 8 + 0 + 0 \\ \text{retval} &= 8 \end{aligned}$$

$$\begin{aligned} \text{foo}(13) &= 13 + 0 + 0 \\ \text{retval} &= 13 \end{aligned}$$

O/p: 3 8 16 13 24 50

When foo is called with a pointer to the root node of the given binary tree, what will it print?



Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE